



UNIVERSIDAD DE LOS ANDES
Facultad de Ingeniería y Ciencias Aplicadas
Arquitectura de Computadores 2026-20

Entrega N°1

Grupo 2:
Josefa Alcaíno Lira
Felipe Izcúe Ferrer
Martín Norton Haigis

Profesor de Cátedra: Jorge Gómez M.

Ayudantes:
Chen Shi Ye
Nicolás Villegas

9 de septiembre de 2026

Resumen

En el presente informe se documenta el diseño e implementación de una calculadora de 4 bits en complemento a dos, construida exclusivamente mediante compuertas lógicas primitivas (**and**, **or**, **not**, **xor**, **nand**, **nor**) y descrita en Verilog estructural. Se presentan las tablas de verdad, mapas de Karnaugh y expresiones booleanas de cada bloque del diseño, junto con los esquemas de implementación en compuertas. La calculadora fue sintetizada y programada exitosamente en una FPGA Lattice iCE40 HX1K (Nandland Go Board), verificando su correcto funcionamiento para las seis operaciones requeridas: reinicio, suma, resta, resta inversa, desplazamiento a la izquierda y desplazamiento a la derecha.

Índice

1. Introducción	4
2. Diseño general y arquitectura	5
2.1. Convenciones de diseño	5
3. Tablas de verdad, mapas de Karnaugh y expresiones booleanas	6
3.1. Sumador completo de 1 bit (full adder)	6
3.1.1. Bloque base del sumador/restador de 4 bits:	6
3.1.2. Mapa de Karnaugh de S :	6
3.1.3. Mapa de Karnaugh de C_{out} :	7
3.2. Sumador/restador de 4 bits compartido	7
3.2.1. Mapas de Karnaugh de las señales de control:	8
3.3. Multiplexor 2:1 (selector de segundo operando)	9
3.3.1. Mapa de Karnaugh del multiplexor:	9
3.4. Barrel shifter	10
3.4.1. Desplazamiento a la izquierda ($A \ll B[1:0]$):	12
3.4.2. Desplazamiento a la derecha ($A \gg B[1:0]$):	12
3.5. Decodificador de operación	13
3.6. Decodificador hexadecimal a siete segmentos	14
3.6.1. Mapa de Karnaugh del segmento a :	16
3.6.2. Mapa de Karnaugh del segmento b :	16
3.6.3. Mapa de Karnaugh del segmento c :	17
3.6.4. Mapa de Karnaugh del segmento d :	17
3.6.5. Mapa de Karnaugh del segmento e :	18
3.6.6. Mapa de Karnaugh del segmento f :	18
3.6.7. Mapa de Karnaugh del segmento g :	19
3.6.8. Implementación que usamos:	19
3.7. Decodificador de signo y cálculo de magnitud	20
3.7.1. Decodificador de signo:	20

3.7.2. Cálculo de magnitud:	20
3.8. Lógica secuencial	21
3.8.1. Latch SR con compuertas NOR:	22
3.8.2. Latch D:	22
3.8.3. Flip-flop D maestro-esclavo:	23
3.8.4. Registro de 4 bits con carga y reinicio:	24
3.9. Filtrado de rebotes y detección de flanco	24
3.9.1. Filtro de rebotes:	24
3.9.2. Detector de flanco:	25
4. Implementación de las operaciones mediante compuertas	26
4.1. Full adder de 1 bit	26
4.2. Multiplexor 2:1	26
4.3. Señales de control (invA, invB, C_{in} , es_reinicio)	27
4.4. Barrel shifter y decodificador de operación	27
5. Conclusiones	29
Declaración de Uso de Herramientas de Inteligencia Artificial	30

1. Introducción

Este informe documenta el diseño e implementación de una calculadora de 4 bits capaz de operar con números representados en complemento a dos, utilizando exclusivamente compuertas lógicas para la lógica combinacional, descrita en Verilog e implementada en una FPGA Lattice iCE40 HX1K (Nandland Go Board).

2. Diseño general y arquitectura

La calculadora recibe un operando **op1** de 4 bits (A), un operando externo **op2** de 4 bits, un selector de 1 bit que elige entre **op2** externo o el resultado de la operación anterior (B), un código de operación de 3 bits, y una señal de confirmar/ejecutar la operación deseada. El resultado se almacena en un registro de 4 bits, cuya salida se realimenta como posible segundo operando de la operación siguiente.

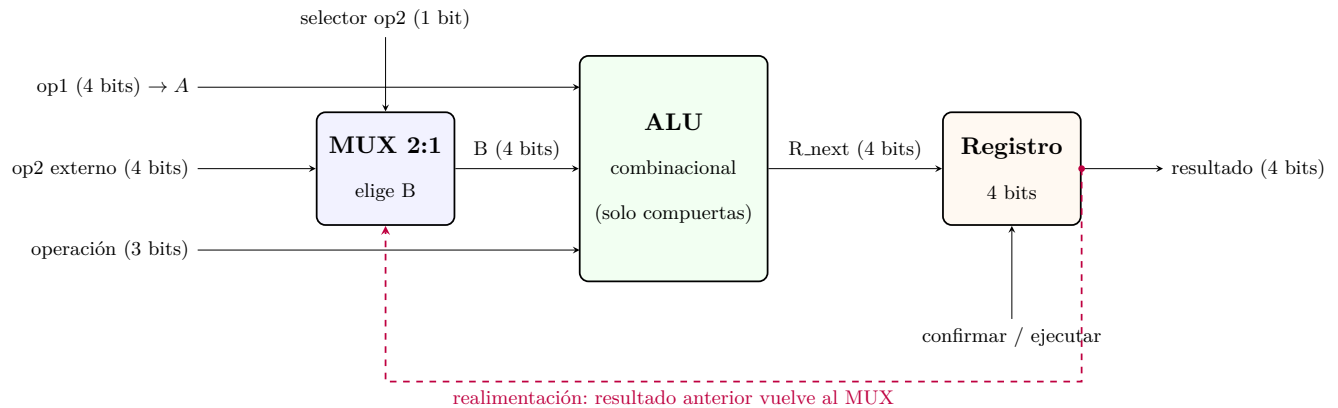


Figura 1: Diagrama de bloques de la calculadora

En el diagrama los **op1** y **op2** (más la realimentación del resultado anterior) entran al MUX 2:1, que junto con el código de operación alimenta la ALU combinacional; el resultado se guarda en el registro al recibir confirmar/ejecutar, y se realimenta como posible segundo operando de la siguiente operación.

2.1. Convenciones de diseño

La calculadora usa un MUX para poder reutilizar el resultado anterior o usar un nuevo número, esto se logra a través de esta operación en los selectores, **op2**: 0 = **op2** externo, **op2**: 1 = resultado anterior.

La señal de confirmación / ejecución depende de un ciclo por pulsación (detecta el flanco de subida luego del debounce)

3. Tablas de verdad, mapas de Karnaugh y expresiones booleanas

En esta sección busca desarrollar, los bloques bases de la calculadora a través de solo expresiones booleanas implementadas en compuertas. A través de primero expresar lo que queremos que haga, su tabla de verdad, sus mapas de Karnaugh y optimización de compuertas.

3.1. Sumador completo de 1 bit (full adder)

3.1.1. Bloque base del sumador/restador de 4 bits:

Recibe A , B , C_{in} y produce la suma S y el acarreo de salida C_{out} .

Cuadro 1: Tabla de verdad del full adder de 1 bit.

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Fuente: Elaboración propia.

3.1.2. Mapa de Karnaugh de S :

filas: C_{in} ; columnas: AB en código Gray.

Cuadro 2: Mapa de Karnaugh de S .

	$AB=00$	$AB=01$	$AB=11$	$AB=10$
$C_{in}=0$	0	1	0	1
$C_{in}=1$	1	0	1	0

Fuente: Elaboración propia.

El patrón de tablero de ajedrez indica que no existen agrupaciones adyacentes posibles, lo cual es característico de una función XOR:

$$S = A \oplus B \oplus C_{in}$$

3.1.3. Mapa de Karnaugh de C_{out} :

Cuadro 3: Mapa de Karnaugh de C_{out} .

	$AB=00$	$AB=01$	$AB=11$	$AB=10$
$C_{in}=0$	0	0	1	0
$C_{in}=1$	0	1	1	1

Fuente: Elaboración propia.

Agrupando los tres pares de unos adyacentes se obtiene la función mayoría:

$$C_{out} = AB + AC_{in} + BC_{in}$$

3.2. Sumador/restador de 4 bits compartido

El sumador de 4 bits se construye encadenando 4 full adders (acarreo en cascada), donde el C_{out} de cada etapa alimenta el C_{in} de la siguiente. El acarreo de salida de la última etapa se descarta, lo que implementa directamente la regla de overflow del enunciado (se conservan solo los 4 bits menos significativos). Para reutilizar el mismo sumador en suma, resta y resta inversa, cada bit de A y B pasa primero por un “inversor controlado” implementado con una compuerta XOR:

$$X' = X \oplus \text{control}$$

que deja pasar X sin cambios si $\text{control}=0$, o lo invierte si $\text{control}=1$. Definiendo invA e invB como las señales de control, y usando el código de operación $OP_2 OP_1 OP_0$ (siendo OP_2 el bit más significativo):

$$\text{invB} = \overline{OP_2} \cdot OP_1 \cdot \overline{OP_0} \quad (\text{activo en resta, } 010)$$

$$\text{invA} = \overline{OP_2} \cdot OP_1 \cdot OP_0 \quad (\text{activo en resta inversa, } 011)$$

$$C_{in} = \text{invA} + \text{invB} = \overline{OP_2} \cdot OP_1$$

Cuadro 4: Señales de control del sumador/restador compartido según la operación.

Operación	invA	invB	C_{in}
Suma ($R = A + B$)	0	0	0
Resta ($R = A - B$)	0	1	1
Resta inversa ($R = B - A$)	1	0	1

Fuente: Elaboración propia.

3.2.1. Mapas de Karnaugh de las señales de control:

Las tres señales dependen de OP_2 , OP_1 y OP_0 . Los tres mapas van juntos en la Tabla 5, con las columnas en código Gray.

Cuadro 5: Mapas de Karnaugh de las señales de Control

Señal		$OP_1 OP_0=00$	01	11	10
invB	$OP_2=0$	0	0	0	1
	$OP_2=1$	0	0	0	0
invA	$OP_2=0$	0	0	1	0
	$OP_2=1$	0	0	0	0
C_{in}	$OP_2=0$	0	0	1	1
	$OP_2=1$	0	0	0	0

Fuente: Elaboración propia.

invA e invB sus expresiones ya son mínimas y no se pueden simplificar. C_{in} es distinto: sus dos unos quedan adyacentes en el mapa, entonces se pueden agrupar de a dos. Al agruparlos, OP_0 cambia de valor dentro del grupo y desaparece de la expresión, que pasa de dos términos de tres literales a uno solo de dos:

$$C_{in} = \overline{OP_2} \cdot OP_1 \cdot \overline{OP_0} + \overline{OP_2} \cdot OP_1 \cdot OP_0 = \overline{OP_2} \cdot OP_1$$

Estas expresiones tienen otra ventaja. En los códigos de shift (100 y 101) y en los no definidos (110 y 111), el factor $\overline{OP_2}$ deja las tres señales en 0 solo. Así no hay que agregar lógica para apagar el sumador, su resultado se descarta después en el decodificador de operación.

3.3. Multiplexor 2:1 (selector de segundo operando)

Selecciona, para cada bit, entre el operando externo `op2` y el resultado de la operación anterior, según el bit `selector_op2` (convención: 0 = externo, 1 = anterior).

Cuadro 6: Tabla de verdad del multiplexor 2

selector_op2	op2 externo	resultado anterior	Salida
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Fuente: Elaboración propia.

3.3.1. Mapa de Karnaugh del multiplexor:

Llamamos s al selector, a al operando externo y b al resultado anterior.

Cuadro 7: Mapa de Karnaugh de la salida del multiplexor 2:1

	$ab=00$	$ab=01$	$ab=11$	$ab=10$
$s=0$	0	0	1	1
$s=1$	0	1	1	0

Fuente: Elaboración propia.

En el mapa hay dos agrupaciones de a dos y ninguna más grande. La primera junta $\{s=0, ab=11\}$ con $\{s=0, ab=10\}$ y elimina b . La segunda junta $\{s=1, ab=01\}$ con $\{s=1, ab=11\}$ y elimina a . Quedan dos términos y no se puede reducir más:

$$\text{Salida} = (\overline{\text{selector_op2}} \cdot \text{op2_externo}) + (\text{selector_op2} \cdot \text{resultado_anterior})$$

Este bloque es chico pero es el que más se repite. Como no podemos usar condicionales, el multiplexor es la única forma de decidir que tenemos, y por eso aparece 354 veces en el circuito completo. Habilitar la carga de un registro, forzar un reinicio, poner prioridad entre dos botones o leer una tabla de verdad se hacen todos con este mismo bloque de cuatro compuertas.

3.4. Barrel shifter

El desplazamiento (`shift left, 100`, y `shift right, 101`) se aplica sobre A (`op1`), en una cantidad dada por los dos bits menos significativos de B : $B[1:0]$. Como A tiene 4 bits, el máximo desplazamiento útil es 3 posiciones, por lo que 2 bits de cantidad son suficientes ($B[1:0] = 00, 01, 10, 11 \rightarrow 0, 1, 2, 3$ posiciones). El desplazamiento es **lógico**: los bits que “entran” se rellenan con 0 (no se preserva el signo).

El bloque se construye como un árbol de 2 etapas de multiplexores 2:1 (el mismo bloque MUX 2:1 ya definido en la sección anterior, reutilizado):

- **Etapla 1:** desplaza 0 o 1 posición, controlada por $B[0]$.
- **Etapla 2:** desplaza 0 o 2 posiciones, controlada por $B[1]$.

Combinando ambas etapas se cubren los 4 desplazamientos posibles (0, 1, 2 y 3), igual que un

contador binario de 2 bits. Cada etapa, para cada bit i , es simplemente:

$$\text{temp}[i] = (\overline{B_0} \cdot a_1) + (B_0 \cdot b_1) \quad R[i] = (\overline{B_1} \cdot \text{temp}[i]) + (B_1 \cdot b_2)$$

donde (a_1, b_1) y (b_2) son las entradas de cada etapa para el bit i , que cambian según la dirección del desplazamiento (izquierda o derecha) y la posición del bit, según se detalla en las tablas siguientes. Cuando la entrada requerida “sale” del registro de 4 bits, se reemplaza por 0 fijo (relleno lógico).

Descomponer el desplazamiento en potencias de dos también sale más barato. Un multiplexor 4:1 por cada bit de salida necesitaría tres bloques base por bit, o sea doce en total, contra los ocho del árbol de dos etapas. La diferencia crece con el ancho: un barrel shifter de N bits usa $N \log_2 N$ multiplexores con este método y N^2 con una matriz completa.

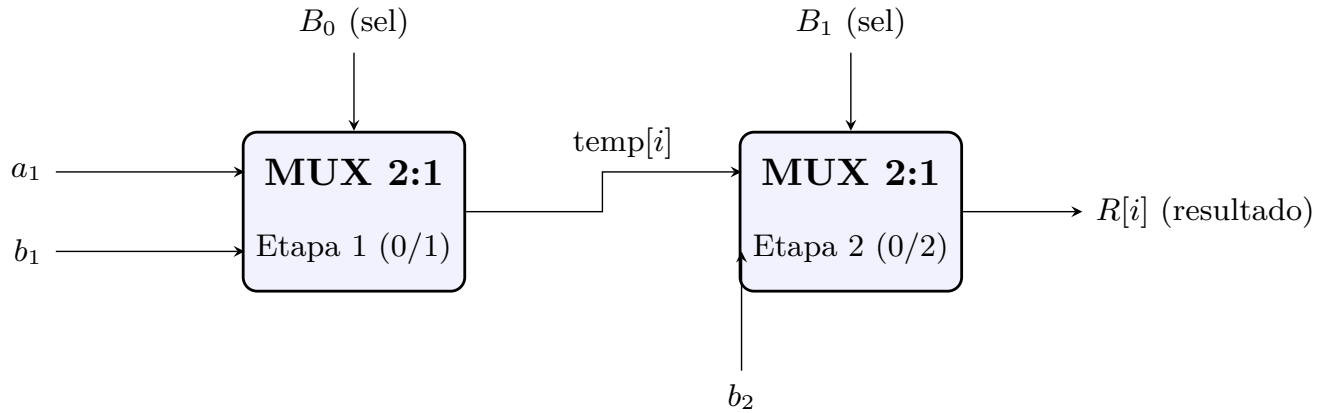


Figura 2: Etapa genérica del barrel shifter (bit i , se repite 4 veces por dirección). Las entradas (a_1, b_1) y b_2 cambian según el bit y la dirección, de acuerdo a las Tablas 8 y 9.

3.4.1. Desplazamiento a la izquierda ($A \ll B[1:0]$):

Cuadro 8: Conexiones del barrel shifter para desplazamiento a la izquierda.

Bit i	a_1 (Etapa 1)	b_1 (Etapa 1)	a_2 (Etapa 2)	b_2 (Etapa 2)
3	$A[3]$	$A[2]$	temp[3]	temp[1]
2	$A[2]$	$A[1]$	temp[2]	temp[0]
1	$A[1]$	$A[0]$	temp[1]	0
0	$A[0]$	0	temp[0]	0

Fuente: Elaboración propia.

A modo de ejemplo, la expresión completa para el bit más significativo del resultado ($R[3]$) queda:

$$R[3] = \overline{B_1} (\overline{B_0} A_3 + B_0 A_2) + B_1 (\overline{B_0} A_1 + B_0 A_0)$$

3.4.2. Desplazamiento a la derecha ($A \gg B[1:0]$):

Cuadro 9: Conexiones del barrel shifter para desplazamiento a la derecha.

Bit i	a_1 (Etapa 1)	b_1 (Etapa 1)	a_2 (Etapa 2)	b_2 (Etapa 2)
3	$A[3]$	0	temp[3]	0
2	$A[2]$	$A[3]$	temp[2]	0
1	$A[1]$	$A[2]$	temp[1]	temp[3]
0	$A[0]$	$A[1]$	temp[0]	temp[2]

Fuente: Elaboración propia.

El desplazamiento a la derecha es al contrario del de la izquierda: en cada etapa, en vez de tomar el bit inferior como entrada de reemplazo, se toma el bit superior (o 0 cuando ya no queda ningún bit superior disponible).

Este diseño produce dos resultados candidatos de 4 bits, uno para **shift left** y otro para **shift right**, que junto con los resultados de las demás operaciones (suma/resta, reinicio) se seleccionan

finalmente en el decodificador de operación.

3.5. Decodificador de operación

Este bloque toma los resultados candidatos ya calculados por los bloques anteriores (el sumador/restador compartido, y los dos barrel shifters) y selecciona cuál de ellos pasa a la salida R , según el código de operación $OP_2 OP_1 OP_0$. Un detalle importante: el sumador/restador compartido, a través de $invA$, $invB$ y C_{in} , ya entrega el resultado correcto para **suma** (001), **resta** (010) y **resta inversa** (011) sin necesidad de un mux adicional. Sin embargo, para **reinicio** (000) las señales de control quedan iguales que en **suma** ($invA = invB = C_{in} = 0$), por lo que el sumador calcularía $A + B$ en vez de 0. Por eso se agrega una señal $es_reinicio$ que fuerza la salida a 0000 únicamente en ese caso.

Cuadro 10: Selección del resultado final según el código de operación.

OP_2	OP_1	OP_0	Operación	Resultado R
0	0	0	Reinicio	0000 (forzado)
0	0	1	Suma	R_{arit}
0	1	0	Resta	R_{arit}
0	1	1	Resta inversa	R_{arit}
1	0	0	Shift left	shift_left
1	0	1	Shift right	shift_right
1	1	X	No definido	—

Fuente: Elaboración propia.

El bloque se arma como un árbol de 2 multiplexores 2:1 (reutilizando otra vez el mismo bloque MUX ya definido) más una señal de bloqueo:

$$R_{shift} = \text{MUX}(\text{shift_left}, \text{shift_right}; OP_0) = (\overline{OP_0} \cdot \text{shift_left}) + (OP_0 \cdot \text{shift_right})$$

$$R_{op2} = \text{MUX}(R_{arit}, R_{shift}; OP_2) = (\overline{OP_2} \cdot R_{arit}) + (OP_2 \cdot R_{shift})$$

$$es_reinicio = \overline{OP_2} \cdot \overline{OP_1} \cdot \overline{OP_0}$$

$$R[i] = R_{op2}[i] \cdot \overline{es_reinicio} \quad \text{para } i = 0, 1, 2, 3$$

Como `es_reinicio` corresponde a un único minterm ($OP_2OP_1OP_0 = 000$), su expresión ya es mínima. La última línea es una compuerta AND con una entrada negada por cada bit del resultado: si `es_reinicio = 1`, fuerza $R[i] = 0$; en cualquier otro código, deja pasar $R_{op2}[i]$ sin cambios.

Que dos multiplexores alcancen para seis operaciones no es casualidad, es porque elegimos así los códigos. OP_2 separa las operaciones aritméticas de las de shift, y OP_0 separa la dirección del shift. Las tres operaciones aritméticas no necesitan ningún multiplexor porque ya salen correctas del mismo sumador. Elegir bien la codificación nos ahorró lógica de decodificación.

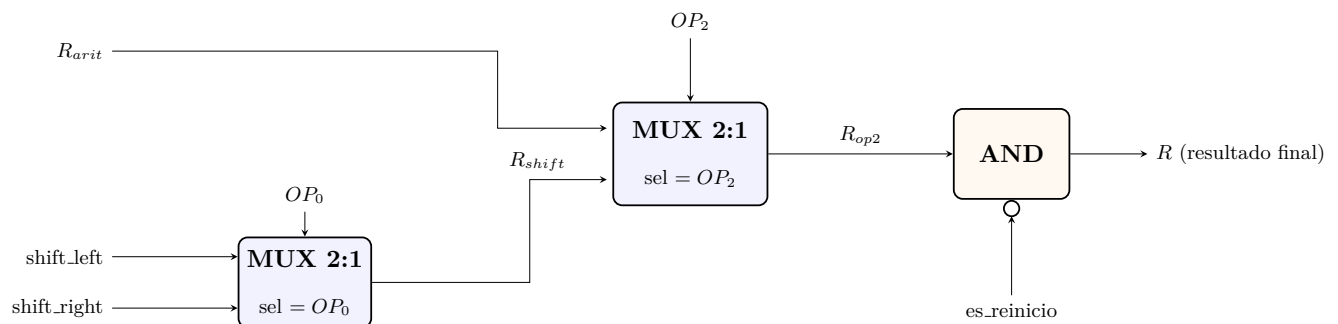


Figura 3: Decodificador de operación

3.6. Decodificador hexadecimal a siete segmentos

El enunciado pide que el segundo display muestre el valor en hexadecimal, entonces hay que traducir un dígito de 4 bits a los siete segmentos del display. Son siete funciones booleanas independientes de cuatro variables, y es la tabla de verdad más grande del proyecto.

Llamamos $D_3D_2D_1D_0$ a los bits del dígito y a hasta g a los segmentos, en la disposición estándar: a arriba, b y c los verticales de la derecha, d abajo, e y f los verticales de la izquierda y g el del medio. La Tabla 11 tiene los dieciséis dígitos. Usamos mayúsculas y minúsculas donde hace falta para poder distinguir b de 8 y d de 0.

Cuadro 11: Tabla de verdad del decodificador hexadecimal a 7 segmentos.

Dígito	D_3	D_2	D_1	D_0	a	b	c	d	e	f	g
0	0	0	0	0	1	1	1	1	1	1	0
1	0	0	0	1	0	1	1	0	0	0	0
2	0	0	1	0	1	1	0	1	1	0	1
3	0	0	1	1	1	1	1	1	0	0	1
4	0	1	0	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	1	0	1	1
6	0	1	1	0	1	0	1	1	1	1	1
7	0	1	1	1	1	1	1	0	0	0	0
8	1	0	0	0	1	1	1	1	1	1	1
9	1	0	0	1	1	1	1	1	0	1	1
A	1	0	1	0	1	1	1	0	1	1	1
b	1	0	1	1	0	0	1	1	1	1	1
C	1	1	0	0	1	0	0	1	1	1	0
d	1	1	0	1	0	1	1	1	1	0	1
E	1	1	1	0	1	0	0	1	1	1	1
F	1	1	1	1	1	0	0	0	1	1	1

Fuente: Elaboración propia.

3.6.1. Mapa de Karnaugh del segmento a :

Cuadro 12: Mapa de Karnaugh del segmento a .

$D_3 D_2 \backslash D_1 D_0$	00	01	11	10
00	1	0	1	1
01	0	1	1	1
11	1	0	1	1
10	1	1	0	1

Fuente: Elaboración propia.

El segmento a tiene 12 unos, en los minterms $\{0, 2, 3, 5, 6, 7, 8, 9, 10, 12, 14, 15\}$. Agrupando queda una suma de productos de 6 términos:

$$a = \overline{D_2} \overline{D_0} + \overline{D_3} D_1 + \overline{D_3} D_2 D_0 + D_2 D_1 + D_3 \overline{D_2} \overline{D_1} + D_3 \overline{D_0}$$

3.6.2. Mapa de Karnaugh del segmento b :

Cuadro 13: Mapa de Karnaugh del segmento b .

$D_3 D_2 \backslash D_1 D_0$	00	01	11	10
00	1	1	1	1
01	1	0	1	0
11	0	1	0	0
10	1	1	0	1

Fuente: Elaboración propia.

El segmento b tiene 10 unos, en los minterms $\{0, 1, 2, 3, 4, 7, 8, 9, 10, 13\}$. Agrupando queda una suma de productos de 5 términos:

$$b = \overline{D_3} \overline{D_2} + \overline{D_3} \overline{D_1} \overline{D_0} + \overline{D_2} \overline{D_0} + \overline{D_3} D_1 D_0 + D_3 \overline{D_1} D_0$$

3.6.3. Mapa de Karnaugh del segmento c :

Cuadro 14: Mapa de Karnaugh del segmento c .

$D_3 D_2 \backslash D_1 D_0$	00	01	11	10
00	1	1	1	0
01	1	1	1	1
11	0	1	0	0
10	1	1	1	1

Fuente: Elaboración propia.

El segmento c tiene 12 unos, en los minterms $\{0, 1, 3, 4, 5, 6, 7, 8, 9, 10, 11, 13\}$. Agrupando queda una suma de productos de 5 términos:

$$c = \overline{D_3} \overline{D_1} + \overline{D_3} D_0 + \overline{D_1} D_0 + \overline{D_3} D_2 + D_3 \overline{D_2}$$

3.6.4. Mapa de Karnaugh del segmento d :

Cuadro 15: Mapa de Karnaugh del segmento d .

$D_3 D_2 \backslash D_1 D_0$	00	01	11	10
00	1	0	1	1
01	0	1	0	1
11	1	1	0	1
10	1	1	1	0

Fuente: Elaboración propia.

El segmento d tiene 11 unos, en los minterms $\{0, 2, 3, 5, 6, 8, 9, 11, 12, 13, 14\}$. Agrupando queda una suma de productos de 5 términos:

$$d = \overline{D_3} \overline{D_2} \overline{D_0} + \overline{D_2} D_1 D_0 + D_2 \overline{D_1} D_0 + D_2 D_1 \overline{D_0} + D_3 \overline{D_1}$$

3.6.5. Mapa de Karnaugh del segmento e :

Cuadro 16: Mapa de Karnaugh del segmento e .

$D_3 D_2 \backslash D_1 D_0$	00	01	11	10
00	1	0	0	1
01	0	0	0	1
11	1	1	1	1
10	1	0	1	1

Fuente: Elaboración propia.

El segmento e tiene 10 unos, en los minterms $\{0, 2, 6, 8, 10, 11, 12, 13, 14, 15\}$. Agrupando queda una suma de productos de 4 términos:

$$e = \overline{D_2} \overline{D_0} + D_1 \overline{D_0} + D_3 D_1 + D_3 D_2$$

3.6.6. Mapa de Karnaugh del segmento f :

Cuadro 17: Mapa de Karnaugh del segmento f .

$D_3 D_2 \backslash D_1 D_0$	00	01	11	10
00	1	0	0	0
01	1	1	0	1
11	1	0	1	1
10	1	1	1	1

Fuente: Elaboración propia.

El segmento f tiene 11 unos, en los minterms $\{0, 4, 5, 6, 8, 9, 10, 11, 12, 14, 15\}$. Agrupando queda una suma de productos de 5 términos:

$$f = \overline{D_1} \overline{D_0} + \overline{D_3} D_2 \overline{D_1} + D_2 \overline{D_0} + D_3 \overline{D_2} + D_3 D_1$$

3.6.7. Mapa de Karnaugh del segmento g :

Cuadro 18: Mapa de Karnaugh del segmento g .

$D_3 D_2 \backslash D_1 D_0$	00	01	11	10
00	0	0	1	1
01	1	1	0	1
11	0	1	1	1
10	1	1	1	1

Fuente: Elaboración propia.

El segmento g tiene 12 unos, en los minterms $\{2, 3, 4, 5, 6, 8, 9, 10, 11, 13, 14, 15\}$. Agrupando queda una suma de productos de 5 términos:

$$g = \overline{D_2} D_1 + D_1 \overline{D_0} + \overline{D_3} D_2 \overline{D_1} + D_3 \overline{D_2} + D_3 D_0$$

3.6.8. Implementación que usamos:

Las siete expresiones de arriba son las mínimas en suma de productos y son la respuesta directa del desarrollo por Karnaugh. Pero el circuito no las usa. Cada segmento se resuelve con un multiplexor de 16 entradas manejado por el dígito, y las dieciséis constantes de cada multiplexor son la columna correspondiente de la Tabla 11.

Lo hicimos así para no equivocarnos. Con el multiplexor la constante es la tabla de verdad, entonces si la tabla está bien el circuito está bien, y no hay ningún paso de transcripción en el medio. Con las expresiones mínimas hay que hacer siete minimizaciones a mano y siete transcripciones a compuertas, y un literal equivocado deja un dígito mal dibujado que solo se ve mirando el display. Como el multiplexor 2:1 ya estaba probado y el árbol de quince instancias no agrega lógica nueva, esta decisión no nos costó nada. Dejamos igual las expresiones mínimas porque muestran el análisis y permiten comparar las dos soluciones.

3.7. Decodificador de signo y cálculo de magnitud

El enunciado pide que el primer display muestre el signo. Como trabajamos en complemento a dos, el signo es directamente el bit más significativo: 0 si el valor es positivo o cero, 1 si es negativo.

3.7.1. Decodificador de signo:

Un display de siete segmentos no puede dibujar un signo más porque le faltan los segmentos verticales del centro. Usamos la convención de los multímetros: el positivo se muestra dejando el display apagado, y el negativo encendiendo solo el segmento g , que dibuja el guion.

Cuadro 19: Tabla de verdad del decodificador de signo

signo	a	b	c	d	e	f	g
0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	1

Fuente: Elaboración propia.

Esta tabla no necesita mapa de Karnaugh. Seis de las siete salidas son constantes y la séptima es la identidad: $g = \text{signo}$ y $a = b = c = d = e = f = 0$. Como los displays de la placa son activos en bajo, en el circuito la salida del segmento g queda como $\overline{\text{signo}}$ y las otras seis fijas en 1.

3.7.2. Cálculo de magnitud:

Al segundo display no le podemos mandar el patrón de bits directo. El decodificador hexadecimal no sabe nada de complemento a dos: si le entra 1001 dibuja un 9, aunque ese patrón valga -7 . La primera versión tenía ese problema, y al recorrer el rango con los botones se veía la secuencia $-8, -9, -A, \dots, -F$ en vez de $-8, -7, -6, \dots, -1$.

Para arreglarlo calculamos la magnitud antes de decodificar. Si el valor es negativo su magnitud es su complemento a dos, y si es positivo o cero es el valor mismo:

$$\text{magnitud} = \begin{cases} \text{valor} & \text{si signo} = 0 \\ \overline{\text{valor}} + 1 = 0 - \text{valor} & \text{si signo} = 1 \end{cases}$$

La operación 0 – valor es la que el sumador/restador ya sabe hacer, con $A = 0000$, $\text{invB} = 1$ y $C_{in} = 1$. Para elegir entre los dos casos usamos un multiplexor 2:1 de 4 bits manejado por el bit de signo. El bloque completo no agrega ninguna compuerta nueva.

Cuadro 20: Verificación del cálculo de magnitud en el rango completo.

Bits	Valor	signo	Magnitud	Se muestra	Sin corregir
0000	0	0	0000	0	0
0001	1	0	0001	1	1
0111	7	0	0111	7	7
1000	-8	1	1000	-8	-8
1001	-7	1	0111	-7	-9
1010	-6	1	0110	-6	-A
1011	-5	1	0101	-5	-b
1100	-4	1	0100	-4	-C
1101	-3	1	0011	-3	-d
1110	-2	1	0010	-2	-E
1111	-1	1	0001	-1	-F

Fuente: Elaboración propia.

El caso de -8 es especial. Su complemento a dos vuelve a dar 1000, porque +8 no cabe en cuatro bits con signo y esa posición no tiene opuesto dentro del rango. Igual funciona: el decodificador lee esos cuatro bits sin signo y dibuja un 8, entonces los dos displays juntos muestran -8, que es el valor correcto. En este caso el resultado sale bien por cómo está armada la representación y no por el cálculo.

3.8. Lógica secuencial

El enunciado restringe la lógica combinacional. Nosotros aplicamos la misma restricción a los elementos de memoria y armamos los biestables con compuertas en vez de instanciar las celdas nativas del chip. Acá desarrollamos esa construcción.

3.8.1. Latch SR con compuertas NOR:

Una compuerta sola no tiene memoria, su salida depende solo de las entradas que tiene en ese momento. Para que un circuito guarde información su salida tiene que volver a la entrada. Un lazo con un número impar de inversiones oscila para siempre, y uno con número par tiene dos estados estables. Si en vez de inversores usamos compuertas NOR conseguimos ese lazo estable y además dos entradas de control: una NOR con la segunda entrada en 0 funciona como inversor, y con la segunda entrada en 1 fuerza su salida a 0.

Cuadro 21: Tabla característica del latch SR con NOR cruzadas.

S	R	$Q(t+1)$	Modo
0	0	$Q(t)$	Mantiene
0	1	0	Reinicio
1	0	1	Set
1	1	—	No permitido

Fuente: Elaboración propia.

3.8.2. Latch D:

$S = R = 1$ no sirve porque deja las dos salidas en 0, que ya no son complementarias, y al soltar las entradas el latch queda en un valor que depende de los retardos físicos. El latch D hace que ese caso no se pueda dar, porque saca las dos señales de una sola entrada de datos y una de habilitación:

$$S = D \cdot EN \qquad R = \overline{D} \cdot EN$$

Como S sale de D y R sale de $\overline{D}$, es imposible que las dos valgan 1 al mismo tiempo. El estado no permitido deja de existir por cómo está armado el circuito, no porque haya que acordarse de evitarlo.

Cuadro 22: Tabla característica del latch D.

EN	D	$Q(t+1)$	Modo
0	X	$Q(t)$	Retiene
1	0	0	Transparente
1	1	1	Transparente

Fuente: Elaboración propia.

3.8.3. Flip-flop D maestro-esclavo:

El latch D es sensible a nivel: mientras la habilitación está activa la salida sigue a la entrada todo el tiempo. Por eso no sirve para hacer contadores. Si la salida vuelve a la entrada pasando por un incrementador y la habilitación está activa, el valor incrementado entra al toque y se vuelve a incrementar. Cuántas vueltas alcanza a dar depende del retardo de las compuertas y de cuánto dure el pulso de habilitación, así que el resultado no es predecible.

La solución es el maestro-esclavo: dos latches D en cascada habilitados con fases opuestas del reloj. El maestro es transparente mientras el reloj está en bajo y el esclavo mientras está en alto, entonces nunca hay un camino transparente entre la entrada y la salida. Cada vuelta del lazo necesita un flanco de reloj.

Cuadro 23: Comportamiento del flip-flop D por flanco de subida.

Instante	Maestro	Esclavo (salida Q)
$CLK = 0$	Transparente, copia D	Retiene
Flanco de subida	Se cierra y congela D	Se abre y publica lo que capturó
$CLK = 1$	Retiene	Transparente, pero su entrada no cambia
Flanco de bajada	Se abre	Se cierra

Fuente: Elaboración propia.

3.8.4. Registro de 4 bits con carga y reinicio:

El flip-flop que armamos no tiene entrada de habilitación ni de reinicio, y su reloj no se puede parar. La habilitación la conseguimos actuando sobre el dato en vez del reloj: un multiplexor elige, en cada flanco, entre el valor nuevo y la propia salida del registro. Cuando le devolvemos la salida, el biestable escribe otra vez el valor que ya tenía, y desde afuera parece que se detuvo. El reinicio lo hace un segundo multiplexor que mete la constante cero.

Cuadro 24: Comportamiento del registro de 4 bits.

reinicio	confirmar	$Q(t+1)$	Cómo se hace
1	X	0000	Multiplexor de reinicio, el último de la cadena
0	1	D_{nuevo}	Multiplexor de carga
0	0	$Q(t)$	El mismo multiplexor, devolviendo Q

Fuente: Elaboración propia.

Que el reinicio le gane a la carga no necesita lógica extra. El multiplexor de reinicio va después en la cadena, entonces su decisión pisa la del anterior. Este mismo patrón, cambiando la cantidad de candidatos, lo usan todos los elementos con estado del diseño: el registro de resultado, el registro de estado, los dos contadores de operandos, el contador de operación y los contadores del filtro de rebotes.

3.9. Filtrado de rebotes y detección de flanco

3.9.1. Filtro de rebotes:

Un botón mecánico no cambia limpio. Sus contactos rebotan durante unos milisegundos y generan decenas de transiciones. Con el reloj de la placa a 25 MHz, cada vez que apretamos el circuito ve una ráfaga de pulsaciones.

Para filtrar eso no medimos cuánto lleva la señal en un valor, sino cuánto lleva sin cambiar. El bloque compara todo el tiempo la señal cruda del pin con la señal ya estable: mientras son distintas un contador sube, y apenas vuelven a coincidir el contador se borra. Recién cuando el contador llega al máximo aceptamos el valor nuevo.

$$\text{diferente} = \text{crudo} \oplus \text{estable}$$

$$\text{limpiar} = \overline{\text{diferente}} + \text{reinicio}$$

$$\text{lleno} = \prod_{i=0}^{11} \text{cnt}[i]$$

$$\text{estable}(t+1) = \text{MUX}(\text{estable}, \text{crudo}; \text{lleno})$$

La señal lleno es el AND de todos los bits del contador, y se activa durante un ciclo justo antes de que el contador dé la vuelta. Con el reloj de la placa el umbral queda en:

$$t_{\text{debounce}} = \frac{4096 \text{ ciclos}}{25 \text{ MHz}} = 163,8 \mu\text{s}$$

3.9.2. Detector de flanco:

Una vez limpia, la señal se queda en alto mientras uno tenga el dedo apoyado, o sea millones de ciclos. Lo que necesita la lógica de control no es que el botón esté apretado, sino que recién se haya apretado. Ese cambio lo detectamos comparando la señal con su propio pasado, usando dos biestables en cascada:

$$\text{pulso} = \text{reg}_1 \cdot \overline{\text{reg}_2}$$

donde reg_1 tiene la señal atrasada un ciclo y reg_2 la atrasada dos. La condición “vale 1 ahora y valía 0 antes” se cumple solo en el ciclo siguiente al flanco de subida.

Usamos dos biestables y no uno por un motivo que no es solo lógico. La versión de un biestable, $\text{pulso} = \text{señal} \cdot \overline{\text{reg}_1}$, funciona igual en teoría, pero mete una señal asíncrona directo a lógica combinacional junto a una señal registrada. La señal viene de un botón y puede cambiar en cualquier momento respecto del reloj, así que eso expone el circuito a fallas por metaestabilidad. Con dos biestables las dos entradas del AND son salidas de registro, y la cadena sirve al mismo tiempo de sincronizador y de detector de flanco.

4. Implementación de las operaciones mediante compuertas

Esta sección traduce cada expresión booleana de la sección anterior a un esquema de compuertas primitivas (**and**, **or**, **not**, **xor**), dibujado con **circuitikz**. Los bloques compuestos (barrel shifter, decodificador de operación) se arman instanciando repetidamente las mismas dos celdas base —full adder y MUX 2:1— por lo que no es necesario volver a dibujarlas compuerta por compuerta cada vez: basta con conectarlas según las tablas ya definidas.

4.1. Full adder de 1 bit

Implementa $S = A \oplus B \oplus C_{in}$ (2 compuertas XOR en cascada) y $C_{out} = AB + AC_{in} + BC_{in}$ (3 compuertas AND más una OR de 3 entradas).

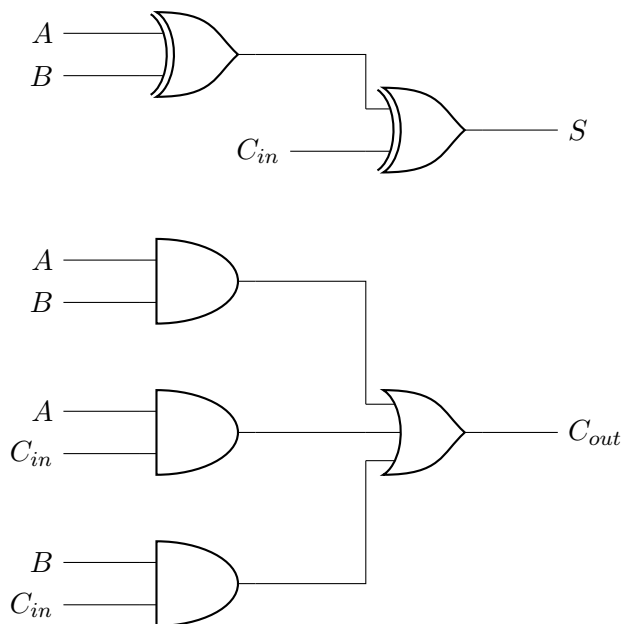


Figura 4: Esquema de compuertas del full adder de 1 bit.

4.2. Multiplexor 2:1

Implementa $Salida = \overline{sel} \cdot a + sel \cdot b$ (1 NOT, 2 AND, 1 OR). Este mismo bloque se reutiliza, sin modificaciones, en el selector de op2, en las dos etapas de cada barrel shifter y en el árbol del decodificador de operación.

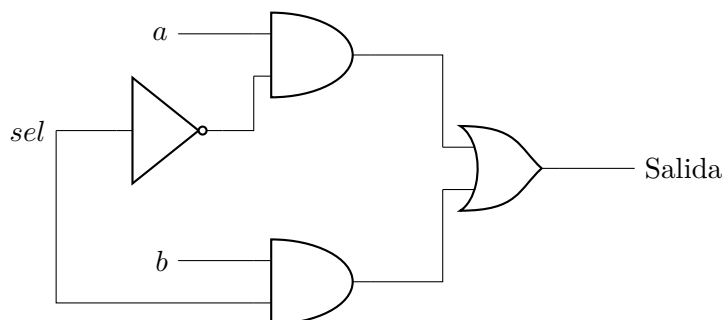


Figura 5: Esquema de compuertas del multiplexor 2:1.

4.3. Señales de control (*invA*, *invB*, C_{in} , *es_reinicio*)

Todas las señales de control derivadas del código de operación siguen el mismo patrón: una compuerta NOT por cada bit que aparece negado en la expresión, seguida de una compuerta AND del tamaño correspondiente. A modo de ejemplo se muestra $es_reinicio = \overline{OP_2} \cdot \overline{OP_1} \cdot \overline{OP_0}$ (3 NOT + 1 AND de 3 entradas); *invA* e *invB* (Tabla 4) se arman igual, solo cambia cuáles entradas llevan NOT antes del AND.

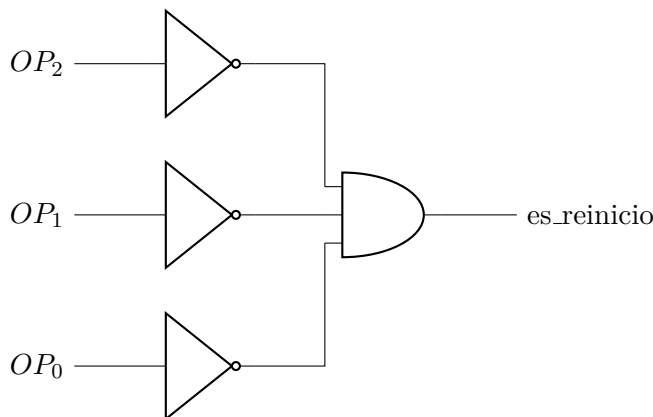


Figura 6: Esquema de compuertas de *es_reinicio*.

4.4. Barrel shifter y decodificador de operación

Ambos bloques se construyen exclusivamente instanciando el MUX 2:1 de la Figura 5 varias veces:

- **Barrel shifter (izquierda/derecha):** 2 etapas $\times$ 4 bits = 8 instancias del MUX 2:1 por

dirección, conectadas según las Tablas 8 y 9.

- **Decodificador de operación:** 2 instancias del MUX 2:1 (una para R_{shift} , $sel = OP_0$; otra para R_{op2} , $sel = OP_2$) por cada uno de los 4 bits del resultado, más una compuerta AND final por bit con $\overline{es_reinicio}$ (Figura 6) como segunda entrada.

No se dibujan compuerta por compuerta porque son repeticiones exactas de la Figura 5; lo único que cambia entre instancias es el cableado de entrada, ya documentado en las tablas correspondientes.

5. Conclusiones

La calculadora de 4 bits fue implementada y verificada exitosamente, tanto en simulación mediante Icarus Verilog y GTKWave, como en la FPGA física. Las seis operaciones requeridas funcionan correctamente, incluyendo el manejo de overflow por truncamiento a 4 bits y la reutilización del resultado anterior como segundo operando.

El principal desafío del proyecto fue la restricción de usar exclusivamente compuertas lógicas primitivas, sin operadores de alto nivel ni estructuras de control de Verilog. Esto obligó a construir desde cero cada elemento del diseño: el sumador de 4 bits como cadena de full adders, los flip-flops como latches maestro-esclavo, el filtro de rebotes como contador de estabilidad, y los displays como multiplexores de 16 entradas. Esta restricción, aunque exigente, permitió comprender en profundidad cómo se implementa la lógica digital a nivel de compuertas.

A nivel práctico, tuvimos que resolver dificultades adicionales relacionadas con el entorno de desarrollo: la configuración de WSL para simulación con Icarus Verilog, la instalación del driver libusbK mediante Zadig para programar la FPGA desde Windows, y el aprendizaje del flujo completo de síntesis con las herramientas de OSS CAD Suite. Superar estos obstáculos fue parte integral del proceso junto con la ejecución misma del proyecto.

Declaración de Uso de Herramientas de Inteligencia Artificial

1. Identificación de la actividad

- **Curso:** Arquitectura de Computadores
- **Código del curso:** 433
- **Nombre de la actividad:** Proyecto N°1
- **Fecha de entrega:** Martes 8 de septiembre
- **Nombre(s) de(los) estudiante(s):** Josefa Alcaíno Lira, Felipe Izcúe Ferrer, Martín Norton Haigis

2. Declaración sobre uso de IA

- ☐ No se utilizaron herramientas de inteligencia artificial en ninguna etapa del desarrollo de esta actividad.
- ☒ Sí se utilizaron herramientas de inteligencia artificial, según se detalla en la sección 3 de este documento.

3. Descripción de uso de IA

Herramienta empleada	Claude (Anthropic): Sonnet 4.6 Medium
Fechas de uso	26 al 30 de agosto - 06 al 08 de septiembre
Estudiantes	Felipe Izcúe Ferrer
Propósito del uso	Análisis completo del informe y de GitHub en búsqueda de puntos faltantes. Apoyo con redacción técnica y adecuada. Explicación de códigos en lenguaje Verilog.
Descripción de integración	Se utilizó Claude para apoyar la escritura y el formato general del informe junto con el repositorio de GitHub. Además, se utilizó para explicar códigos de la calculadora, explicando funcionamiento y formato.

Herramienta empleada	Claude (Anthropic): Sonnet 5 High
Fechas de uso	26 de agosto - 07 y 08 de septiembre
Estudiantes	Josefa Alcaíno Lira
Propósito del uso	Generación de códigos en lenguaje Verilog para la creación de la calculadora. Explicación de estos mismos. Apoyo sobre los códigos y funcionamiento general para desarrollo del informe y repositorio de GitHub.
Descripción de integración	Se utilizó Claude como apoyo para generar los códigos y ordenar ideas durante la redacción del informe y repositorio, y para revisar y pulir la redacción de los desarrollos.

Herramienta empleada	Claude (Anthropic): Opus 5 (Ultracode)
Fechas de uso	28 de agosto y del 5-7 de septiembre
Estudiantes	Martín Norton Haigis
Propósito del uso	Apoyo en la estructura del informe, explicación para el entendimiento profundo del lenguaje Verilog y, por último, la corrección ortográfica y de puntuación.
Descripción de integración	Se utilizó este modelo para entender y modelar en profundidad la arquitectura de la calculadora, cómo ordenar el informe respecto a la estructura de la calculadora, y las correspondientes correcciones de gramática y puntuación.

4. Declaración de responsabilidad académica

Los firmantes de este documento declaramos que:

- Hemos revisado críticamente todo el contenido entregado.
- Comprendemos plenamente el trabajo presentado y somos capaces de explicarlo y defenderlo académicamente.
- Asumimos responsabilidad por la calidad, veracidad y originalidad del contenido entregado,

independientemente del uso o no uso de herramientas de inteligencia artificial.

- La información declarada en este documento es veraz.



Josefa Alcaíno Lira



Felipe Izcúe Ferrer



Martín Norton Haigis